

CHAPTER : 7

★ "QUEUES (FIFO)" ★

QUEUES :-

Definition :-

"A queue is a data structure which is just like a real life queue. The data which enters the queue first, leaves first. Only the data at front of queue is directly accessible."

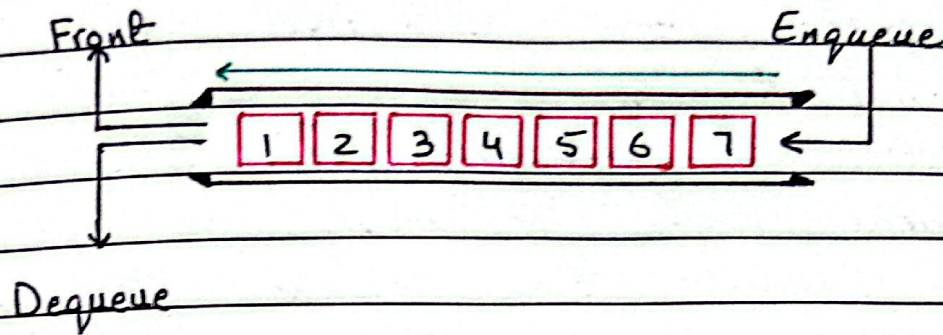
Explanation -

Imagine the example of people waiting in a queue at the reception of a movie theatre, the person that entered the queue first will be dealt first, then the one after him and so on. This protocol is called "First In First Out" (FIFO), and this is the working principle of "queue". Some real life examples would be printing documents or scanning them as well.

Date: / /

Day

MEMORY DIAGRAM:-



As you might have already noticed by now, a queue is basically the opposite of a stack in its working principle.
"FIFO is opposite of LIFO"

Now we will learn how to implement a queue using a prior data structure which is "linked list".

QUEUE USING LIST:-

Now let's form FIFO protocol by using a list. There are three main methods in a queue -

- 1) Enqueue/push "Enters data through rear in queue"
- 2) front "Gives data at front of queue"
- 3) Dequeue/pop "Remove data at front of queue"

Now let's relate a list with a queue.

If we use push-back as the enqueue^{up}, front as front and pop-front as a dequeue, our list will basically become a queue and of course we have to remove the access to any methods of a list which can violate the "FIFO" protocol. Now after you have this understanding, let's look at the code of a queue using list:-

Code :-

```
class queue {  
private:  
    std::list<int> l;  
public:  
    void enqueue(int n) { l.push_back(n); }  
    void dequeue() { l.pop_front(); }  
    int front() { return l.front(); }  
    bool empty() { return l.empty(); }  
    int size() { return l.size(); }  
};
```

In order for this to work, you must include STL list using -
#include <list>

Date: 1/1/

Day

This was too easy wasn't it?
Now let's also try implementing
a queue using our own list
instead of C++ STL :-

For this we need a `ListNode`
class just like the one we
learned in chapter 4 of linked lists.
After that, let's customize our
queue class to be compatible
with this `ListNode` class.

Code :-

```
class queue{  
private:  
    ListNode* head, *tail;  
public:  
    Queue(){ head = tail = nullptr; }  
    // other methods like enqueue etc.  
};
```

Now let's look at general
methods of a queue like
enqueue, front and dequeue.
They will be a copy of
push-back, front and pop-front
of a linked list which
you already know but will

Date: _/ _/ _

also implement some exception handling in these:-

ENQUEUE:-

Enqueue is just like a push-back of a list, it inserts a value from the rear end of a queue:-

Code:-

```
void enqueue(int val){
    ListNode* newNode = new ListNode(val);
    if(!head)
        head = tail = newNode;
    else
    {
        tail->next = newNode;
        tail = newNode;
    }
}
```

No exception handling is required here so there isn't much of anything new here. Next, let's look at front method:-

FRONT :-

Front method returns the value of head Node and if the list is empty then we use exception handling by throwing an error.

Code :-

```
int front() {  
    if (!head) { throw std::runtime_error("Empty"); }  
    else  
        return head->data;  
}
```

This allows to make sure that front data is only returned if list is not empty. This is a great example of exception handling by throwing error.

After this we are left with one main function which is dequeue or pop. Which is like a pop-front but again let's implement it with our own ListNode.

DEQUEUE :-

If the list is empty, our dequeue will throw the same error as the front method, otherwise it will remove the data at front of queue like a pop-front :-

Code :-

```
void dequeue() {  
    if (!head) throw std::runtime_error("Empty");  
    else  
    {  
        ListNode* temp = head;  
        head = head->next;  
        delete temp;  
    }  
}
```

This is all for the implementing methods of a queue for now, by now you shall have a good understanding about the internal working of a queue, now let's move on to some practice exercises.

PRACTICE EXERCISE :-

- 1) The queue which we made in this chapter using our own `ListNode` class has a memory leak, it creates dynamic `ListNode`s but never deletes them, design a destructor which deletes all of these `ListNode`s.
- 2) Create a function called `reverseQueue`, which takes a "`queue<int> q`", that is passed by reference, this function should return void and the queue's data shall be reversed. (Use of C++ STL is allowed.)
- 3) What are the time complexities of `enqueue/push`, `front` and `dequeue/pop` methods.

As always, all codes of this chapter are available on my github at:

["github.com/coder-Retro/Fast-University"](https://github.com/coder-Retro/Fast-University)

This was all about queues for now, nextup we will dive into a special type of queue called deque or "Double Ended Queue", it will be easy, Till Then Good Luck!